CodeAlpha (DevOps) task : 02

TITLE : Web Server using Docker

Aim: To learn Docker containerization basics, deploy and manage a web server inside Docker containers, understand container lifecycle management, monitor container health, troubleshoot issues, and explore container-based application deployment.

Objective

- Learn Docker containerization basics.
- Deploy and manage a web server inside Docker containers.
- Understand container lifecycle and commands.
- Monitor container health and troubleshoot issues.
- Explore container-based application deployment best practices.

Software Requirements

- Windows 11 Operating System
- Docker Desktop
- Command Prompt
- Internet Connection
- Web Browser (Chrome/Edge)

Procedure

Step 1: Docker Installation

Docker Desktop was downloaded and installed on the system. Initially, Docker showed a WSL (Windows Subsystem for Linux) update error. The issue was resolved by updating WSL and restarting Docker Desktop.

Verification command:

docker --version

Result:

Docker was successfully installed and configured.

Step 2: Testing Docker Installation

To verify Docker functionality, the following command was executed:

docker run hello-world

Result:

The container executed successfully and displayed the "Hello from Docker!" message, confirming that Docker Engine was working correctly.

Step 3: Downloading Nginx Image

The official Nginx web server image was downloaded from Docker Hub using:

docker pull nginx

Initially, network and TLS handshake timeout errors occurred while downloading the image. The issue was resolved by authenticating with Docker Hub.

Docker login command:

docker login

After successful authentication, the Nginx image was downloaded successfully.

Step 4: Verifying Downloaded Images

The downloaded images were verified using:

docker images

Result:

The Nginx image was successfully available in the local Docker repository.

Step 5: Deploying Nginx Web Server Container

A Docker container was created and started using:

docker run -d -p 8080:80 --name mywebserver nginx

Explanation:

- docker run → Creates and runs a container.
- -d → Runs container in detached mode.
- -p 8080:80 → Maps host port 8080 to container port 80.
- --name mywebserver → Assigns a custom container name.
- nginx → Docker image used.

Result:

Nginx web server container was deployed successfully.

Step 6: Checking Running Containers

The running container was verified using:

docker ps

Result:

The Nginx container appeared in the running container list.

Step 7: Accessing the Web Server

The web server was accessed through a browser using:

http://localhost:8080

Result:

The default Nginx welcome page was displayed successfully, confirming that the web server was operational.

Step 8: Understanding Container Lifecycle

Different Docker lifecycle commands were executed to manage the container.

Stop Container

docker stop mywebserver

Start Container

docker start mywebserver

Restart Container

docker restart mywebserver

Remove Container

docker rm -f mywebserver

Result:

Container lifecycle operations were successfully performed.

Step 9: Monitoring and Troubleshooting

View Container Logs

docker logs mywebserver

Purpose:

Displays container-generated logs for debugging and troubleshooting.

Monitor Resource Usage

docker stats

Purpose:

Displays real-time CPU, memory, and network usage statistics.

Inspect Container Details

docker inspect mywebserver

Purpose:

Provides detailed configuration and runtime information about the container.

Result:

Container health and performance were successfully monitored.

Step 10: Docker Hub Authentication and Troubleshooting

During deployment, several issues were encountered:

Issue 1

WSL update error prevented Docker Desktop from starting.

Solution:

WSL was updated and Docker Desktop was restarted.

Issue 2

TLS Handshake Timeout while downloading Nginx image.

Solution:

- DNS cache was flushed.
- Docker Desktop was restarted.
- Docker Hub authentication was performed using:

docker login

Result:

Image download completed successfully.

Docker Commands Used

docker --version
docker run hello-world
docker login
docker pull nginx
docker images
docker run -d -p 8080:80 --name mywebserver nginx
docker ps
docker stop mywebserver
docker start mywebserver
docker restart mywebserver
docker logs mywebserver
docker stats

docker inspect mywebserver

docker rm -f mywebserver

Outcome

- Docker Desktop was installed and configured successfully.
- Docker Engine functionality was verified.
- Nginx web server image was downloaded from Docker Hub.
- A web server was deployed successfully inside a Docker container.
- Container lifecycle operations were performed successfully.
- Container monitoring and troubleshooting techniques were learned.
- Docker Hub authentication and networking issues were resolved.
- Practical knowledge of containerized application deployment was gained.

Conclusion

This experiment successfully demonstrated Docker containerization concepts by deploying an Nginx web server inside a Docker container. Various Docker commands were used to create, manage, monitor, and troubleshoot containers. The task provided practical exposure to modern container-based application deployment and management techniques used in DevOps environments.